

Chapitre 5

Grammaires non contextuelles



Le chapitre 2 nous a permis de travailler sur une classe de langages dont l'expressivité permet par exemple de faire de la recherche de motifs (préfixes, suffixes, facteurs...). L'expressivité de la classe des langages réguliers est cependant limitée et on a facilement mis en évidence des langages qui ne correspondent pas à cette classe. Il faut avoir en tête qu'un des objectifs majeurs de la théorie des langages est de permettre l'analyse automatique d'un code produit dans un langage de programmation donné. Si les langages réguliers permettent de faire ce qu'on appelle l'analyse lexicale c'est-à-dire la reconnaissance des différentes chaînes de caractères utilisées par le programmeur : elle permet notamment de reconnaître les "mots clés" du langage comme `for`, `while`, `struct`... les commentaires, une variable (d'où des règles) ou encore une constante numérique. Les langages réguliers ne permettront cependant pas de comprendre la structure de votre code afin de l'interpréter en une suite d'instructions machines élémentaires. Ce sont les grammaires non contextuelles qui vont permettre cette interprétation de la structure du code que l'on appellera l'analyse syntaxique.

1 Un premier exemple

Nous allons commencer par une question qui apparaît naturellement dans l'analyse d'un programme : l'interprétation du parenthésage.

Nous allons donc considérer le langage des expressions bien parenthésées sur l'alphabet suivant : $\Sigma = \{ (,) \}$.

Une expression bien parenthésée est définie inductivement de la manière suivante :

- ε est bien parenthésée
- Si e et e' sont bien parenthésées, alors $e.e'$ l'est aussi.
- Si e est bien parenthésée alors (e) l'est aussi.

Les grammaires vont d'ailleurs parfaitement correspondre à l'expressivité de formules que l'on peut définir inductivement.

Voici la description de la grammaire que nous allons considérer :

$$S \rightarrow \varepsilon | SS | (S)$$

On part du symbole S et tant qu'il reste S dans les productions, on peut appliquer une des règles ci-dessus, par exemple :

$$\begin{aligned} S &\Rightarrow SS \\ &\Rightarrow (S)S \\ &\Rightarrow (S)SS \\ &\Rightarrow (SS)SS \\ &\Rightarrow ((S)S)SS \\ &\Rightarrow (())SS \\ &\Rightarrow (())(S)S \\ &\Rightarrow (())(S)(S) \\ &\Rightarrow (())()S \\ &\Rightarrow (())()SS \\ &\dots \\ &\Rightarrow (())()() \end{aligned}$$

➤ On remarquera au passage que le lemme de l'étoile nous permet de montrer que ce langage n'est pas régulier.

2 Définitions

Définition - Grammaire non contextuelle

Une **grammaire non contextuelle** est un 4-uplet $\mathcal{G} = \{\mathcal{V}, \Sigma, \mathcal{R}, S\}$ où :

- $\mathcal{V}$ est un ensemble de symboles appelés *symboles non terminaux*.
- Σ est un ensemble de symboles appelés *symboles terminaux*, on aura alors $\Sigma \cap \mathcal{V} = \emptyset$
- S est un élément de $\mathcal{V}$ appelé *symbole initial*.
- $\mathcal{R} \subset \mathcal{V} \times (\Sigma \cup \mathcal{V})^*$ est appelé *ensemble de règles de production*. C'est un ensemble fini.

EXEMPLE. Dans l'exemple de la partie précédente, $\mathcal{V} = \{S\}$, $\Sigma = \{ (,) \}$ et $\mathcal{R} = \{(S, \varepsilon), (S, SS), (S, (S))\}$

Définition - Dérivation directe

Soit $\mathcal{G} = \{\mathcal{V}, \Sigma, \mathcal{R}, S\}$ une grammaire non contextuelle et soit $u = u_1 X u_2 \in (\Sigma \cup \mathcal{V})^* \mathcal{V} (\Sigma \cup \mathcal{V})^*$.

On dit que u **se dérive directement** en v si il existe une règle de la forme $X \rightarrow s$ telle que $v = u_1 s u_2$.

On note alors $u \Rightarrow v$.

Définition - Dérivation gauche ou droite

Soit $\mathcal{G} = \{\mathcal{V}, \Sigma, \mathcal{R}, S\}$ une grammaire non contextuelle et soit $u = u_1 X u_2 \in (\Sigma \cup \mathcal{V})^* \mathcal{V} (\Sigma \cup \mathcal{V})^*$.

On dit que u **se dérive immédiatement à gauche** (resp. à droite) en v si $u \Rightarrow v$ et $u_1 \in \Sigma^*$ (resp. $u_2 \in \Sigma^*$).

Autrement dit, on a choisi d'appliquer une règle de production au symbole non terminal le plus à gauche (resp. le plus à droite).

EXEMPLE. Considérons : $S \rightarrow SGa|b$; $G \rightarrow GSb|a$

Dérivation quelconque

$S \Rightarrow SGa$
 $\Rightarrow SGSba$
 $\Rightarrow SGaGSba$
 $\Rightarrow bGaGSba$
 $\Rightarrow bGaaSba$
 $\Rightarrow bGaabba$
 $\Rightarrow baaabba$

Dérivation gauche

$S \Rightarrow SGa$
 $\Rightarrow bGa$
 $\Rightarrow bGSba$
 $\Rightarrow bGSbSba$
 $\Rightarrow baSbSba$
 $\Rightarrow babbSba$
 $\Rightarrow babbbba$

Dérivation droite

$S \Rightarrow SGa$
 $\Rightarrow Saa$
 $\Rightarrow SGaaa$
 $\Rightarrow Saaaaa$
 $\Rightarrow SGaaaaaa$
 $\Rightarrow Saaaaaaa$
 $\Rightarrow baaaaaaa$

➤ On remarque qu'il y a des choix à faire ...

Définition - Dérivation

Soit $\mathcal{G} = \{\mathcal{V}, \Sigma, \mathcal{R}, S\}$ une grammaire non contextuelle.

On note $\Rightarrow^*$ la cloture réflexive transitive de $\Rightarrow$.

On dit que u se dérive en v quand $u \Rightarrow^* v$, c'est-à-dire quand il existe une suite $u = u_0, \dots, u_n = v$ d'éléments de $(\Sigma \cup \mathcal{V})^*$ telle que $\forall i \in [0; n-1], u_i \Rightarrow u_{i+1}$.

Une suite de dérivations immédiates est alors appelée une dérivation.

De plus, si toutes les dérivations immédiates utilisées sont gauches (resp. droites) on dit que la dérivation est gauche (resp. droite).

Définition - Langage engendré par une grammaire

Soit $\mathcal{G} = \{\mathcal{V}, \Sigma, \mathcal{R}, S\}$ une grammaire non contextuelle.

On définit le **langage engendré par cette grammaire** comme l'ensemble des mots $u \in \Sigma^*$ pour lesquelles il existe une dérivation depuis le symbole initial S .

En d'autres termes, en le notant $\mathcal{L}(\mathcal{G})$, on a :

$$\mathcal{L}(\mathcal{G}) = \{u \in \Sigma^* \mid S \Rightarrow^* u\}$$

Définition - Langage non contextuel

Un langage engendré par une grammaire non contextuelle est un langage non contextuel.

➤ Mais pourquoi parle-t-on de grammaire "non contextuelle"?

En fait, il s'agit d'une mauvaise traduction... On ferait mieux de dire "hors-contexte" (ou *context-free* en anglais). En effet, ce terme convient plus car dans une grammaire avec la règle $A \rightarrow \alpha$ (où $A \in \mathcal{V}, \alpha \in \Sigma$), on peut dériver A en α **indépendamment du contexte dans lequel ce symbole apparaît**. En d'autres termes, la dérivation des symboles non-terminaux ne dépend pas des symboles qui entourent ces derniers.

3 Des exemples

Nous allons décrire les grammaires qui correspondent aux langages suivants :

1. $L = \{a^n b^n \mid n \in \mathbb{N}\}$

On considère alors la grammaire $S \rightarrow S | aSb | \epsilon$

2. L'ensemble des palindromes sur l'alphabet $\{a, b\}$ (**exemple de preuve formelle à connaître**).

On considère la grammaire $\mathcal{G}_1$ avec $S \rightarrow S | aSa | bSb | a | b | \epsilon$. Notons $\mathcal{P}$ l'ensemble des palindromes.

➤ $\mathcal{P} \subset \mathcal{L}(\mathcal{G}_1)$: Montrons par récurrence sur $n \in \mathbb{N}$, tout palindrome de longueur n est dans $\mathcal{L}(\mathcal{G}_1)$.

- $n = 0$: $\epsilon \in \mathcal{L}(\mathcal{G}_1)$.
- $n = 1$: $a, b \in \mathcal{L}(\mathcal{G}_1)$
- $H_{n-1}, H_n \Rightarrow H_{n+1}$:
 - 1er cas : Il existe $p' \in \mathcal{P}$ tq $p = ap'a$ avec $|p'| = n - 1$.
Par H.R, $S \Rightarrow^* p'$, donc $S \Rightarrow aSa \Rightarrow^* ap'a$.
 - 2ème cas : Il existe $p' \in \mathcal{P}$ tq $p = bp'b$ avec $|p'| = n - 1$.
Par H.R, $S \Rightarrow^* p'$, donc $S \Rightarrow bSb \Rightarrow^* bp'b$.

D'où la première inclusion.

➤ $\mathcal{L}(\mathcal{G}_1) \subset \mathcal{P}$: On montre par récurrence sur $n \in \mathbb{N}^*$ que s'il existe une dérivation de longueur n qui engendre u alors $u \in \mathcal{P}$.

- $n = 1$: alors $u = \epsilon$ ou a ou b . Dans tous les cas, $u \in \mathcal{P}$.
- $H_n \Rightarrow H_{n+1}$: Soit $u \in \Sigma^*$ pour lequel on a une dérivation de longueur $n + 1$.

$$S \Rightarrow \begin{cases} aSa \\ \text{ou} \\ bSb \end{cases} \Rightarrow^n u$$

Sans perte de généralité, supposons que l'on ait affaire à la dérivation initiale $S \Rightarrow aSa$.

Alors, $u = au'a$ et u' est engendré en n étapes. Par H.R, u' est un palindrome. On en conclut donc que u est un palindrome.

d'où l'inclusion réciproque.

« *finito* » - Augustine, 2024

3. L'ensemble des formules logiques du calcul propositionnel.

On saisit la grammaire $S \rightarrow \perp | \top | \neg S | S \wedge S | S \vee S$

4. Considérons la grammaire suivante : $E \rightarrow I|E + E|E * E|(E)$; $I \rightarrow a|b|Ia|Ib|I0|I1$.

Montrer que $(aa10 + bbb) * (a + b0)$ est dans le langage engendré par cette grammaire de symbole initial E .
On a :

$$\begin{aligned}
 E &\Rightarrow E * E \\
 &\Rightarrow E * (E) \\
 &\Rightarrow E * (E + E) \\
 &\Rightarrow^2 E * (I + I0) \\
 &\Rightarrow^2 E * (a + b0) \\
 &\Rightarrow (E) * (a + b0) \\
 &\Rightarrow (E + E) * (a + b0) \\
 &\Rightarrow^2 (I + I) * (a + b0) \\
 &\Rightarrow^2 (I0 + Ib) * (a + b0) \\
 &\Rightarrow^2 (I10 + Ibb) * (a + b0) \\
 &\Rightarrow^3 (aa10 + bbb) * (a + b0)
 \end{aligned}$$

d'où le mot recherché.

5. Considérons maintenant la grammaire suivante :

- $\mathcal{V} = \{S, I, E, V, N, C\}$
- $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \text{print}, =, +, (,), ;, x, y\}$
- $\mathcal{R}$:

$$\begin{aligned}
 S &\rightarrow I; S | \varepsilon \\
 I &\rightarrow V = E | \text{print}(E) \\
 E &\rightarrow E + E | N \\
 V &\rightarrow x | y \\
 N &\rightarrow CN | C \\
 C &\rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
 \end{aligned}$$

Montrer que le mot $x = 42$ appartient au langage engendré par cette grammaire.

On a :

$$\begin{aligned}
 S &\Rightarrow I; S \\
 &\Rightarrow V = E; S \\
 &\Rightarrow V = E; \\
 &\Rightarrow V = N; \\
 &\Rightarrow V = CN; \\
 &\Rightarrow V = CC; \\
 &\Rightarrow x = CC; \\
 &\Rightarrow x = C2; \\
 &\Rightarrow x = 42
 \end{aligned}$$

4 REC $\subset$ CFL

Théorème

L'ensemble des langages réguliers est inclus dans l'ensemble des langages non contextuels.

DÉMONSTRATIONS. (les deux sont à connaître!)

1. Il suffit de montrer que $\emptyset \in \text{CFL}$, $\forall a \in \Sigma, \{a\} \in \text{CFL}$ et que CFL est stable par union, concaténation et étoile de Kleene.

- (a) On prend la grammaire $S \rightarrow S$ et on a bien $\emptyset \in \text{CFL}$.
- (b) Soit $a \in \Sigma$, en prenant $S \rightarrow a$, on a bien $\{a\} \in \text{CFL}$.
- (c) Soient

$$\mathcal{G}_1 = \{\mathcal{V}_1, \Sigma_1, \mathcal{R}_1, S_1\}$$

$$\mathcal{G}_2 = \{\mathcal{V}_2, \Sigma_2, \mathcal{R}_2, S_2\}$$

avec $\mathcal{V}_1 \cap \mathcal{V}_2 = \emptyset$.

On construit :

$$\mathcal{G} = (\mathcal{V}_1 \cup \mathcal{V}_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, \mathcal{R}, S)$$

avec $S \notin \mathcal{V}_1 \cup \mathcal{V}_2$ et $\mathcal{R} = \{S \rightarrow S_1, S \rightarrow S_2\} \cup \mathcal{R}_1 \cup \mathcal{R}_2$

On a alors $\mathcal{L}(\mathcal{G}) = \mathcal{L}(\mathcal{G}_1) + \mathcal{L}(\mathcal{G}_2) \in \text{CFL}$.

(d) Avec les notations précédentes, on construit la grammaire $\mathcal{G}'$:

$$\mathcal{G}' = (\mathcal{V}_1 \cup \mathcal{V}_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, \mathcal{R}', S)$$

avec $S \notin \mathcal{V}_1 \cup \mathcal{V}_2$ et $\mathcal{R} = \{S \rightarrow S_1 S_2\} \cup \mathcal{R}_1 \cup \mathcal{R}_2$

⚠ On a besoin de $\mathcal{V}_1 \cap \mathcal{V}_2 = \emptyset$!

On a alors $\mathcal{L}(\mathcal{G}') = \mathcal{L}(\mathcal{G}_1)\mathcal{L}(\mathcal{G}_2) \in \text{CFL}$.

(e) Soit $\mathcal{G} = \{\mathcal{V}, \Sigma, \mathcal{R}, S\}$ une grammaire, on pose

$$\mathcal{G}'' = \{\mathcal{V} \cup \{S_0\}, \Sigma \cup \{\varepsilon\}, \mathcal{R}'', S_0\}$$

avec $S_0 \notin \mathcal{V}, \mathcal{R}'' = \{S_0 \rightarrow \varepsilon, S_0 \rightarrow SS_0\}$

On conclut quant à la minimalité de REC.

➤ La classe CFL est stable par union, concaténation et étoile mais elle ne le sera pas pour l'intersection, le complémentaire ni la différence symétrique!

2. En prenant un AFD quelconque A , il faut réussir à construire une grammaire $\mathcal{G}$ telle que $\mathcal{L}(A) = \mathcal{L}(\mathcal{G})$.

Soit donc $A = (\Sigma, Q, q_0, F, \delta)$ un AFD quelconque. Construisons donc une telle grammaire.

On pose

$$\mathcal{G} = (\{X_q \mid q \in Q\}, \Sigma, \mathcal{R}, X_{q_0})$$

$$\text{avec } \mathcal{R} = \left(\bigcup_{q \in Q, a \in \Sigma} \{X_q \rightarrow aX_{\delta(q,a)}\} \right) \cup \left(\bigcup_{q \in F} \{X_q \rightarrow \varepsilon\} \right)$$

On peut alors montrer par récurrence sur $|m|$ que :

$$\forall q, q' \in Q, \delta^*(q, m) = q' \Leftrightarrow X_q \Rightarrow^* mX_{q'}$$

On en déduit alors que :

$$\begin{aligned} m \in \mathcal{L}(A) &\Leftrightarrow \delta^*(q_0, m) \in F \\ &\Leftrightarrow \exists q_f \in F, X_{q_0} \Rightarrow^* mX_{q_f} \\ &\Leftrightarrow X_{q_0} \Rightarrow^* m \quad (\text{car seuls les états } q' \in F \text{ ont une règle } X_{q'} \rightarrow \varepsilon) \end{aligned}$$

5 Arbre de dérivation

La plupart du temps, un mot généré par une grammaire admet plusieurs dérivations distinctes.

Par exemple, si l'on considère la grammaire $S \rightarrow \neg S|(S)|S \vee S|S \wedge S|a|b|c$, les trois dérivations suivantes correspondent au même mot $m = a \wedge b \vee c$:

$$\begin{aligned} S &\Rightarrow S \wedge S \\ &\Rightarrow a \wedge S \\ &\Rightarrow a \wedge S \vee S \\ &\Rightarrow a \wedge b \vee S \\ &\Rightarrow a \wedge b \vee c \end{aligned}$$

$$\begin{aligned} S &\Rightarrow S \wedge S \\ &\Rightarrow S \wedge S \vee S \\ &\Rightarrow S \wedge S \vee c \\ &\Rightarrow S \wedge b \vee c \\ &\Rightarrow a \wedge b \vee c \end{aligned}$$

$$\begin{aligned} S &\Rightarrow S \vee S \\ &\Rightarrow S \vee c \\ &\Rightarrow S \wedge S \vee c \\ &\Rightarrow a \wedge S \vee c \\ &\Rightarrow a \wedge b \vee c \end{aligned}$$

En un certain sens, que nous allons préciser, les deux premières dérivations ne sont "pas vraiment différentes" : depuis $S \wedge S$, on peut comment par remplacer le premier S et obtenir $a \wedge S$ ou commencer par le deuxième et obtenir $S \wedge S \vee S$, mais "ça commute". La troisième dérivation, en revanche, est complètement différente, parce qu'elle ne correspond pas au même *arbre syntaxique*.

Définition - Arbre syntaxique

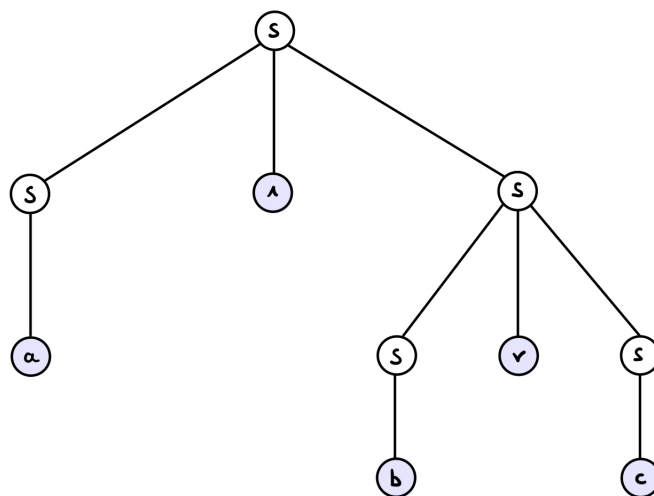
Soit $\mathcal{G} = \{\mathcal{V}, \Sigma, \mathcal{R}, S\}$ une grammaire non contextuelle. On va associer à chaque dérivation dans $\mathcal{G}$ un arbre de la manière suivante :

- La racine de l'arbre est S .
- Les noeuds internes sont des éléments de $\mathcal{V}$.
- Les feuilles sont des éléments de $\Sigma \cup \{\varepsilon\}$.
- Toute feuille d'étiquette ε est fille unique.
- Chaque noeud interne X qui a pour fils $u_1, \dots, u_n$ est tel que $X \rightarrow u_1 \dots u_n$ est une règle de la grammaire.
- La concaténation des feuilles de gauche à droite est le mot engendré par la dérivation.

EXEMPLES.

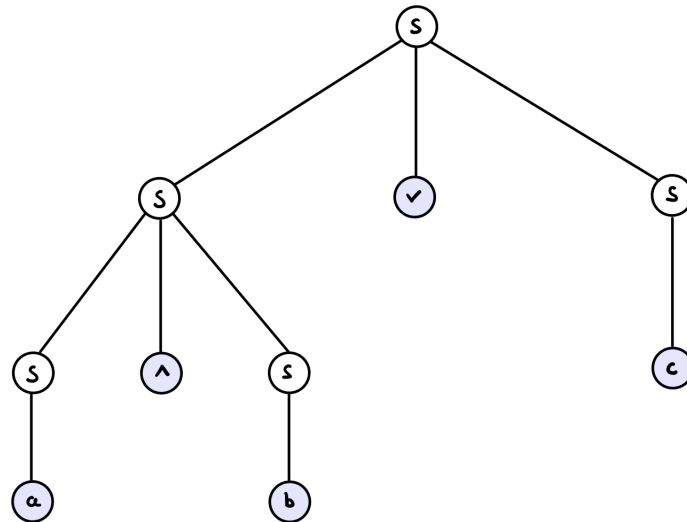
1. Reprenons les trois dérivations précédentes :

(a) On a



(b) On dispose donc (comme mentionné précédemment) du même arbre.

(c) On a



2. On dispose de la grammaire suivante :

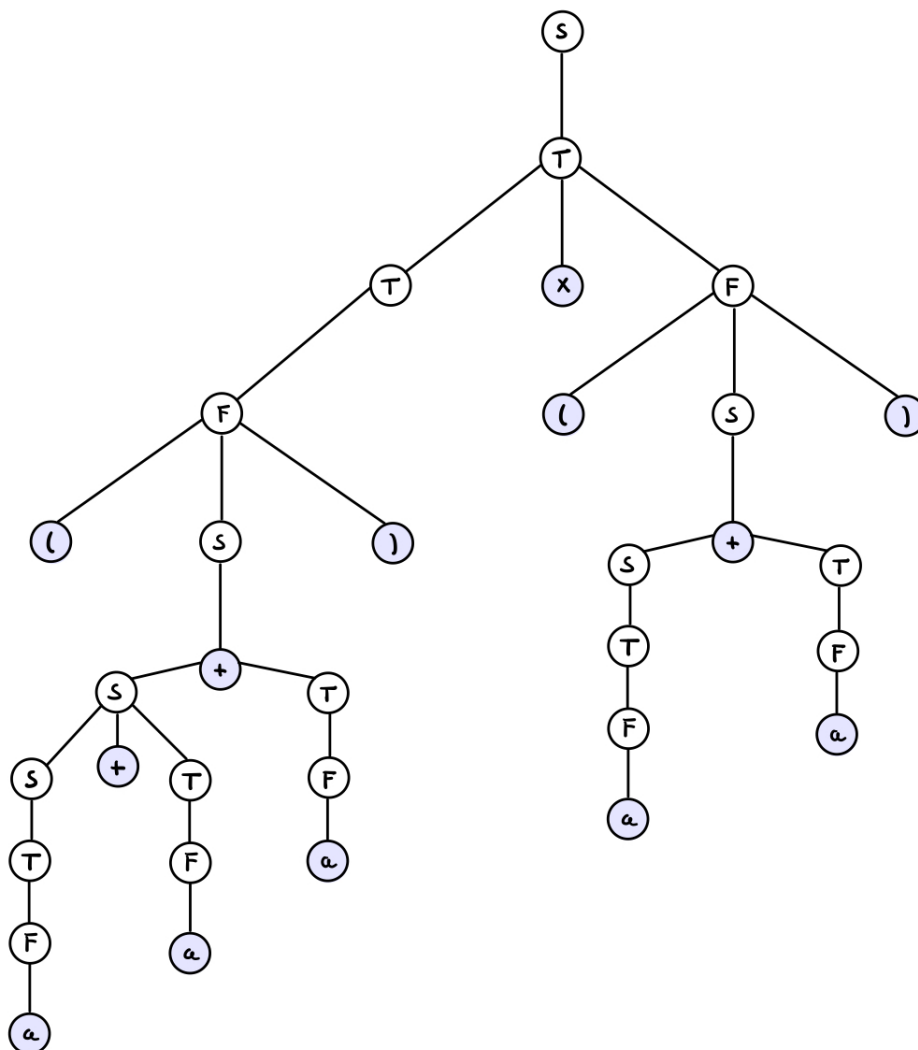
$$S \rightarrow S + T \mid T$$

$$T \rightarrow T \times F \mid F$$

$$F \rightarrow (S) \mid a$$

Construire un arbre pour $(a + a + a) \times (a + a)$.

On a :



On veut maintenant formaliser le fait que deux dérivations soient ou non *réellement différentes*, signifiant alors qu'elles ne correspondent pas au même arbre syntaxique.

Proposition - Une affaire de bijections

Soient $\mathcal{G}$ une grammaire et $u \in \mathcal{L}(\mathcal{G})$. Alors,
il y a une bijection entre les dérivations gauches de u et les arbres de dérivation de u .

À partir de l'arbre, c'est en fait un parcours préfixe de l'arbre (de gauche à droite donc) qui permet d'obtenir la dérivation gauche.

➤ De même, il y a une bijection entre les dérivations droites de u et ses arbres de dérivation.

DÉMONSTRATION.

À un arbre de dérivation T (dont la racine est un symbole $\alpha \in \Sigma \cup \mathcal{V}$ quelconque), on associe la dérivation $d(T)$ définie comme suit :

- Si T est une feuille, alors $d(T) = \alpha$ (dérivation de longueur nulle)
- Sinon, $T = (X; \alpha_1 \dots \alpha_k)$ avec $X \rightarrow \alpha_1 \dots \alpha_k$ une règle de G et $\alpha_1 \dots \alpha_k \in \Sigma \cup \mathcal{V}$. On a alors des dérivations $d(\alpha_i) : \alpha_i \Rightarrow^* u_i$, que l'on "concatène" : $d(T) = X \Rightarrow \alpha_1 \dots \alpha_k \Rightarrow^* u_1 \alpha_2 \dots \alpha_k \Rightarrow^* \dots \Rightarrow^* u_1 \dots u_k$.

Autrement dit, on applique les règles dans l'ordre du parcours préfixe de l'arbre. On remarque alors que :

- En prenant T un arbre de dérivation de u , on obtient bien une dérivation de u (la racine de T est S , et les feuilles lues dans l'ordre donnent bien le mot u).
- Il s'agit bien, par construction, d'une dérivation gauche.

Inversement, à une dérivation gauche $S \Rightarrow^* \alpha \in (\Sigma \cup \mathcal{V})^*$, on associe un arbre de dérivation partiel, dont les feuilles sont étiquetées par des symboles quelconques et donnent α quand on les lit de gauche à droite, comme suit :

- Si la dérivation est de longueur nulle, alors on prend l'arbre réduit à S .
- Sinon, la dérivation est de la forme $S \Rightarrow^* uX\gamma \Rightarrow u\beta\gamma$ avec $u \in \Sigma^*$, $\beta, \gamma \in (\Sigma \cup \mathcal{V})^*$ et $X \rightarrow \beta$ dans $\mathcal{R}$. On prend alors l'arbre de dérivation associé à $S \Rightarrow^* uX\gamma$ et l'on remplace la feuille X la plus à gauche par $(X; \beta_1, \dots, \beta_k)$

Ces deux applications sont bien réciproques l'une de l'autre.

Définition - Grammaire ambiguë

Soient $\mathcal{G}$ une grammaire et $u \in \mathcal{L}(\mathcal{G})$.

- On dit que u est **ambigu** pour $\mathcal{G}$ s'il existe plusieurs arbres de dérivation distincts pour u .
- On dit que la grammaire $\mathcal{G}$ est **ambiguë** s'il existe au moins un mot u ambigu pour $\mathcal{G}$.

➤ Ici, l'**ambiguïté** est une propriété de la grammaire et non du langage.

Définition - Grammaires faiblement équivalentes

Deux grammaires $\mathcal{G}$ et $\mathcal{G}'$ sont dites **faiblement équivalentes** si $\mathcal{L}(\mathcal{G}) = \mathcal{L}(\mathcal{G}')$.

- Le faiblement vient du fait qu'on ne demande que l'égalité des langages générés, et rien sur les dérivations ou les arbres syntaxiques.
- Il n'y a pas vraiment de manière canonique de définir une équivalence forte.
- Une grammaire ambiguë peut être faiblement équivalente à une grammaire non ambiguë ou non.

Théorème

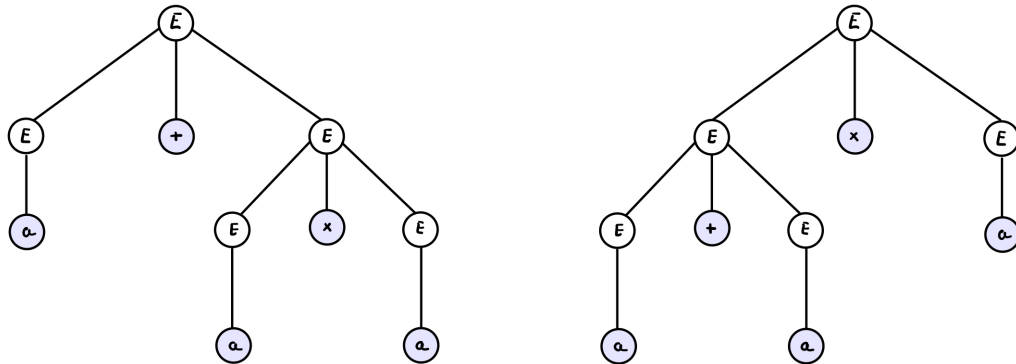
Soient $\mathcal{G}$ une grammaire et $u \in \mathcal{L}(\mathcal{G})$.

Les propositions suivantes sont équivalentes :

1. u est ambigu pour $\mathcal{G}$.
2. u admet plusieurs dérivations gauches pour $\mathcal{G}$.
3. u admet plusieurs dérivations droites pour $\mathcal{G}$.

EXEMPLES.

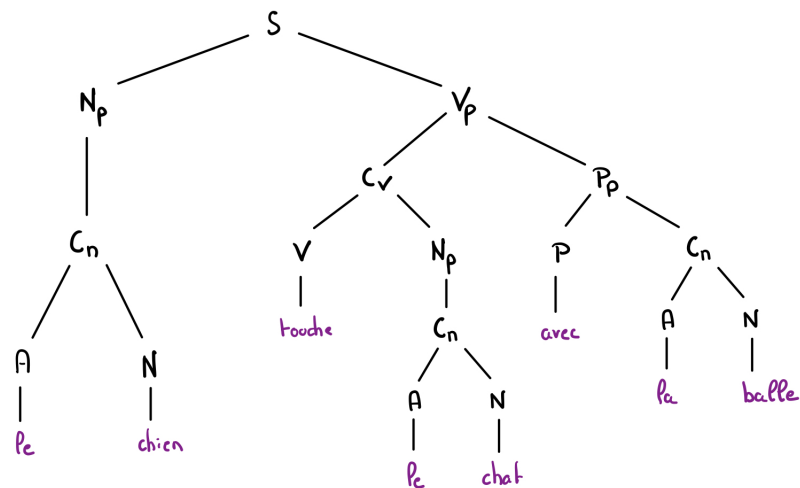
1. Considérons $E \rightarrow E + E | E \times E | (E) | a$ de symbole initial E . Montrer que cette grammaire est ambiguë. On considère le mot $a + a \times a$. On obtient alors les deux arbres de dérivations différents suivants (prouvant donc l'ambiguïté) :



2. Considérons la phrase "le chien touche le chat avec la balle" dans la grammaire suivante :

$$\begin{aligned}
 S &\rightarrow N_p V_p \\
 N_p &\rightarrow C_n | C_n P_p \\
 C_n &\rightarrow A N \\
 P_p &\rightarrow P C_n \\
 C_v &\rightarrow V | V N_p \\
 V_p &\rightarrow C_v | C_v P_p \\
 A &\rightarrow \text{un} | \text{le} | \text{la} \\
 V &\rightarrow \text{joue} | \text{voie} | \text{touche} \\
 N &\rightarrow \text{chien} | \text{chat} | \text{balle} \\
 P &\rightarrow \text{avec}
 \end{aligned}$$

On a alors l'arbre de dérivation suivant :



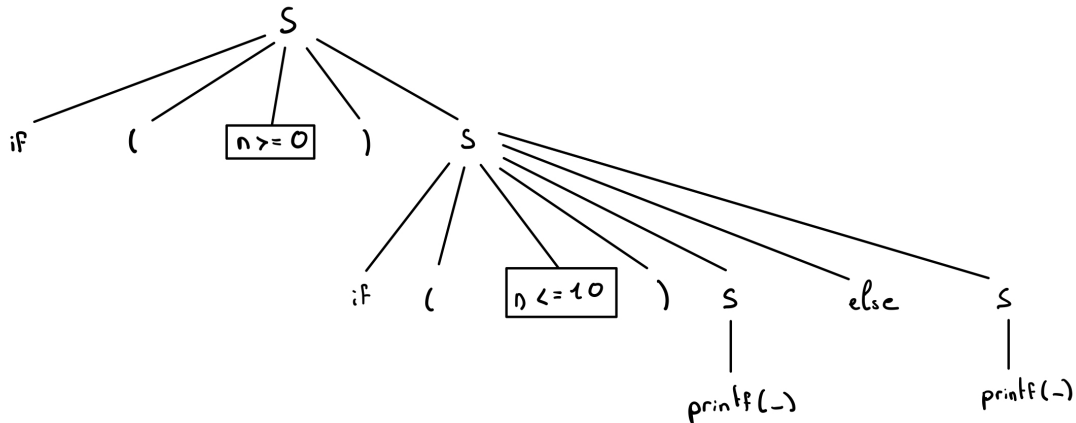
3. Un exemple classique d'ambiguïté dans les grammaires est celui du *dangling else*. En C, comme dans beaucoup de langages, la clause `else` est optionnelle dans un `if/then/else`.

Les deux morceaux de code suivants sont en fait syntaxiquement corrects :

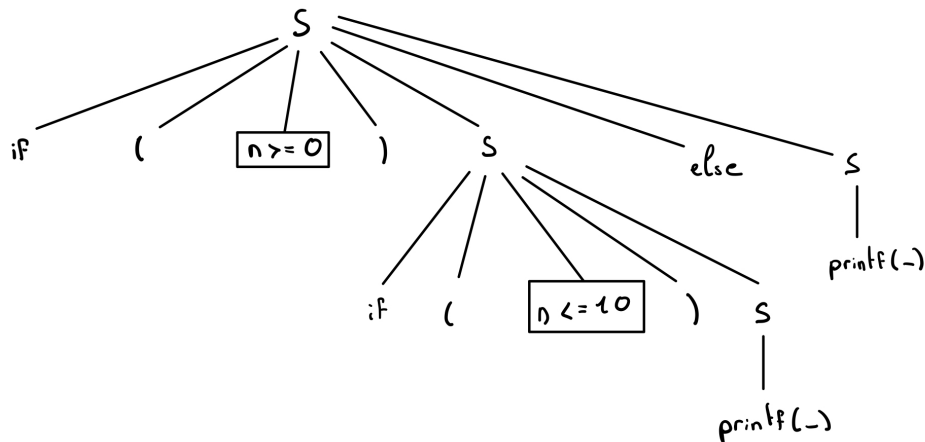
```
1 if (n >= 0)
2   if (n <= 10)
3     printf("0 <= n <= 10 \n");
4   else
5     printf("n > 10 \n");
```

```
1 if (n >= 0)
2   if (n <= 10)
3     printf("0 <= n <= 10 \n");
4   else
5     printf("n < 0 \n");
```

Intuitivement, ils correspondent aux deux arbres syntaxiques suivants :



et



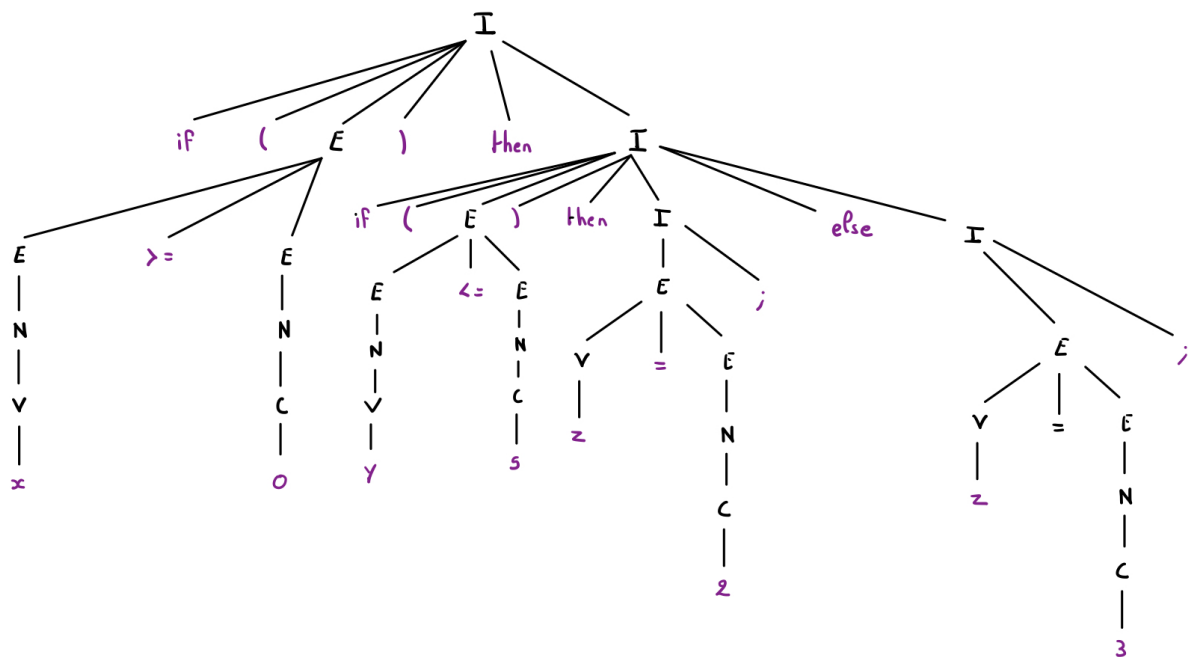
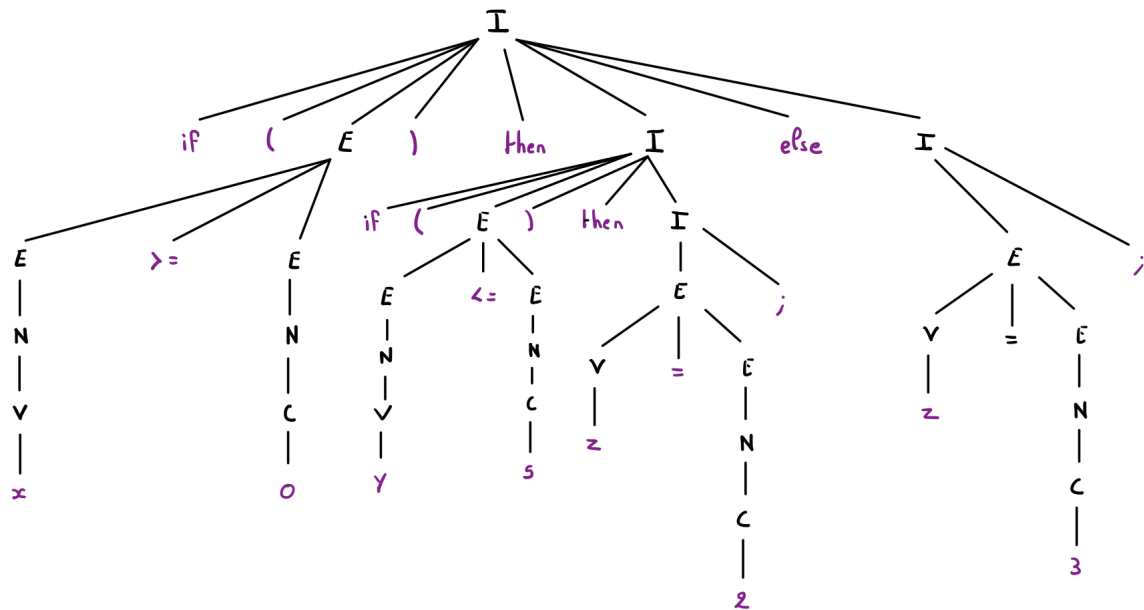
➤ Le problème est que, à l'exception du contenu du deuxième `printf`, il s'agit en fait de deux fois le même morceau de code : l'indentation est ignorée par le compilateur en C. Pour lever l'ambiguïté, il y a trois solutions :

- garder une grammaire ambiguë, mais préciser le choix qui sera fait lors de l'analyse syntaxique (en l'occurrence, c'est le premier arbre qui sera produit) .
- réécrire la grammaire pour supprimer l'ambiguïté, tout en gardant la même syntaxe pour le langage de programmation (on remplace la grammaire par une grammaire faiblement équivalente).
- modifier la syntaxe du langage (les règles d'indentation de Python, par exemple, évitent ce problème).

4. Voici un exemple de grammaire permettant de visualiser le problème :

$$\begin{aligned} I &\rightarrow \text{if}(E) \text{ then } I \mid \text{if}(E) \text{ then } I \text{ else } I \mid E; \\ E &\rightarrow V = E \mid E <= E \mid E >= E \mid N \\ N &\rightarrow CN \mid C \mid V \\ C &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \\ V &\rightarrow x \mid y \mid z \end{aligned}$$

Considérons le mot suivant `if (x>=0) then if (y <=5) then z=2; else z=3;.`
On dispose alors des deux arbres de dérivation suivants :



6 Analyse syntaxique

Il s'agit de la théorie algorithmique qui consiste à construire un arbre d'analyse d'un mot dans une grammaire donnée. C'est une des étapes fondamentales de la compilation qui permet de construire l'arbre qui capture la structure de votre code. Les techniques sont hors-programme.

Exemples ad-hoc d'algo permettant de générer un arbre de dérivation pour un mot reconnu par la grammaire suivante :

1. On considère la grammaire suivante :

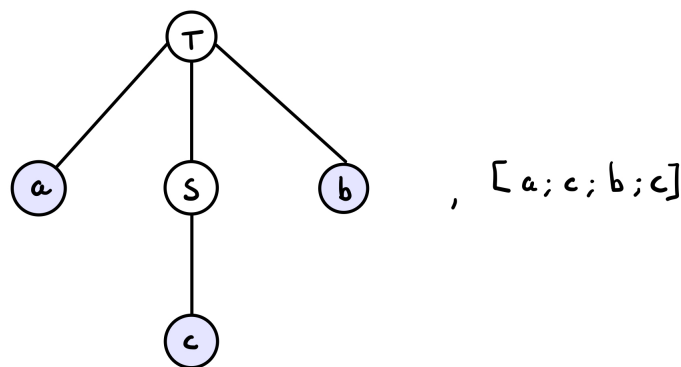
$$S \rightarrow TS \mid c$$

$$T \rightarrow aSb$$

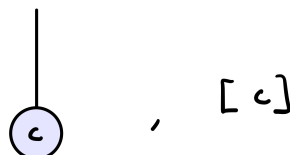
On écrit donc deux fonctions `genereS` et `genereT` ayant la spécification suivante :

- `genereS` renvoie un arbre qui correspond à l'arbre de dérivation du plus grand préfixe de l engendré par S
- `genereT` renvoie un arbre qui correspond à l'arbre de dérivation du plus grand préfixe de l engendré par T

Par exemple, `genereT [a;c;b;a;c;b;c]` renvoie



et `genereS [c,c]` donne



On obtient donc le code suivant :

```

1 type arbred = A | B | C | S of arbred * arbred | T of arbred * arbred * arbred | Sb of arbred
2
3 let rec genereS (l : char list) : arbred * char list =
4   match l with
5   | 'c'::t -> Sb(C) , t
6   | h::t -> let a1,l1 = genereT l in
7               let a2,l2 = genereS l1 in
8               S(a1,a2),l2
9   | _ -> failwith "Syntax error"
10 and genereT (l : char list) : arbred * char list =
11   match l with
12   | 'a'::t -> ( let a1,l1 = genereS t in
13                 match l1 with
14                 | 'b'::s -> T(A,a1,B),s
15                 | _ -> failwith "Syntax error"
16               )
17   | _ -> failwith "Syntax error"

```

➤ On remarque qu'avec une règle $S \rightarrow ST$ on ne peut pas adapter la méthode.

2. On considère la grammaire suivante :

$$S \rightarrow T + S \mid T$$

$$T \rightarrow F \times T \mid F$$

$$F \rightarrow (S) \mid a$$

De même, on va créer trois fonctions : `genereS`, `genereT` et `genereF`.

```
1 type arbre = A | Plus of arbre * arbre | Fois of arbre * arbre
2
3 let rec genereS (l : char list) : arbre * char list =
4     let aT,lT = genereT l in
5     match lT with
6     | '+'::t -> let aS,lS = genereS t in Plus(aT,aS),lS
7     | _ -> aT,lT
8
9 and genereT (l : char list) : arbre * char list =
10    let aF,lF = genereF l in
11    match lF with
12    | '*'::t -> let aT,lT = genereT t in Fois(aF,aT),lT
13    | _ -> aF,lF
14
15 and genereF (l : char list) : arbre * char list =
16    match l with
17    | 'a'::s -> A,s
18    | '('::s -> (let aS,lS = genereS s in
19        match lS with
20        | ')'::q -> aS,q
21        | _ -> failwith "Parenthese non-fermee !"
22    )
23    | _ -> failwith "Syntax error"
```